



Programación Genética

Inteligencia Artificial y Mini-Robots (2023251)

Diego Fernando Bustos Horta, dbustosh@unal.edu.co

Julián Mateo Vargas Riveros, jvargasri@unal.edu.co

2026-10-05

1. Ejercicios de programación genética

Se desarrollaron los ejercicios 1 y 3 mediante programación genética (PG) en Python con la biblioteca DEAP. A continuación se presenta el diseño, la implementación y los resultados del codificador de siete segmentos.

1.1. Ejercicio 1: codificador de siete segmentos

El circuito recibe un dígito BCD representado por cuatro bits, $b_3b_2b_1b_0$, y produce las siete salidas $a, \dots, g$. Se adoptó un visualizador de cátodo común, por lo que un valor de 1 enciende el segmento. Para las entradas no válidas en BCD, de 1010 a 1111, se definió la salida 0000000.

La tabla de verdad constituye el criterio con el cual se evalúa cada programa generado. En cada salida, los bits aparecen en el orden $abcdefg$:

Decimal	$b_3b_2b_1b_0$	$abcdefg$
0	0000	1111110
1	0001	0110000
2	0010	1101101
3	0011	1111001
4	0100	0110011
5	0101	1011011
6	0110	1011111
7	0111	1110000
8	1000	1111111
9	1001	1111011
10–15	1010–1111	0000000

El conjunto de terminales fue $T = \{b_3, b_2, b_1, b_0, 0, 1\}$. El conjunto de funciones incluyó las compuertas AND, OR, XOR y NOT, además de un multiplexor. Este último devuelve u cuando la condición s es verdadera y v en caso contrario:

$$\text{MUX}(s, u, v) = \begin{cases} u, & s = 1, \\ v, & s = 0. \end{cases}$$

En el código, las operaciones de NumPy permiten evaluar simultáneamente un árbol sobre las 16 entradas posibles:

```

1 conjunto = gp.PrimitiveSet("BCD", 4)
2 conjunto.renameArguments(
3     ARG0="b3", ARG1="b2", ARG2="b1", ARG3="b0"
4 )
5
6 conjunto.addPrimitive(np.logical_and, 2, name="AND")
7 conjunto.addPrimitive(np.logical_or, 2, name="OR")

```

```

8 conjunto.addPrimitive(np.logical_xor, 2, name="XOR")
9 conjunto.addPrimitive(np.logical_not, 1, name="NOT")
10 conjunto.addPrimitive(
11     lambda s, u, v: np.where(s, u, v), 3, name="MUX"
12 )
13
14 conjunto.addTerminal(True, name="ONE")
15 conjunto.addTerminal(False, name="ZERO")

```

Listing 1: Funciones y terminales disponibles para los árboles booleanos.

Se evolucionó un árbol independiente para cada segmento. Así, un individuo candidato para a es una expresión booleana que recibe los cuatro bits y devuelve únicamente el estado del segmento a . Su aptitud depende del número de discrepancias con la columna correspondiente de la tabla de verdad:

$$f_k(P) = 1 - \frac{1}{16} \sum_{i=0}^{15} \mathbf{1}[P(b_i) \neq y_{i,k}] - 0,00005 |P|,$$

donde k identifica el segmento, $y_{i,k}$ es la salida esperada para la entrada i , y $|P|$ es el número de nodos del árbol. El último término favorece árboles más pequeños cuando su comportamiento es similar.

```

1 resultado = predecir(gp.compile(arbol, conjunto), entradas)
2 errores = np.count_nonzero(
3     resultado != esperadas[:, indice]
4 )
5 aptitud = 1 - errores / 16 - 0.00005 * len(arbol)

```

Listing 2: Evaluación de la aptitud de un árbol para un segmento.

Se utilizaron 350 individuos y 135 generaciones por segmento. La selección se hizo mediante torneos de cuatro individuos; los dos mejores pasaron directamente a la siguiente generación. La probabilidad de cruce fue 0,78, la de mutación fue 0,14 por descendiente y la altura máxima permitida para los árboles fue siete. Se fijaron semillas para reproducir cada corrida.

La salida de la ejecución muestra, para cada segmento, cuántas de las 16 entradas se clasificaron incorrectamente, el número de nodos del mejor árbol y su expresión. Por ejemplo, `XOR(b3, OR(...))` se lee desde la función exterior hacia sus argumentos; ONE representa la constante lógica 1.

```

Segmento a: 1 errores; 10 nodos; árbol = XOR(b3, OR(NOT(OR(b0, b3)), OR(b1, b2)))
Segmento b: 0 errores; 13 nodos; árbol = NOT(MUX(OR(b0, b3), MUX(b1, b3, b2), MUX(b1, b2, b3)))
Segmento c: 0 errores; 11 nodos; árbol = MUX(MUX(b3, b2, b0), NOT(b3), MUX(b1, b2, ONE))
Segmento d: 1 errores; 17 nodos; árbol = XOR(OR(AND(b1, b3), NOT(OR(AND(b2, b0), AND(b3, b2)))), XOR(b1, NOT(b3)))
Segmento e: 0 errores; 7 nodos; árbol = NOT(OR(b0, MUX(b1, b3, b2)))
Segmento f: 1 errores; 9 nodos; árbol = NOT(MUX(b2, OR(b3, AND(b0, b1)), b1))
Segmento g: 1 errores; 5 nodos; árbol = XOR(b3, OR(b1, b2))
Errores antes de corregir: 4
Corrección d: entrada 0, 0 -> 1; OR M_0
Corrección f: entrada 1, 1 -> 0; AND NOT M_1
Corrección a: entrada 4, 1 -> 0; AND NOT M_4
Corrección g: entrada 7, 1 -> 0; AND NOT M_7
Verificación final: 112/112 salidas correctas

```

Listing 3: Resultados de la evolución y verificación final.

Los segmentos b , c y e acertaron directamente sus 16 casos. Cada uno de los otros cuatro árboles cometió un solo error; por eso la consola informa **Errores antes de corregir: 4**. Por ejemplo, el árbol de e corresponde a

$$e = \neg(b_0 \vee \text{MUX}(b_1, b_3, b_2))$$

y fue exacto. En cambio, el árbol de g corresponde a

$$g_{PG} = b_3 \oplus (b_1 \vee b_2).$$

Para la entrada decimal 7, este árbol devuelve 1, aunque la tabla exige 0.

Se completó el circuito mediante un mintermino M_i para cada entrada errónea. M_i vale 1 únicamente cuando los cuatro bits corresponden al número i . Si un segmento debía estar encendido, se añadió el mintermino mediante OR; si debía estar apagado, se lo excluyó mediante AND NOT. El siguiente fragmento realiza ambas operaciones:

```

1 for i, segmento in zip(*np.where(prediccion != esperadas)):
2     nuevo = int(esperadas[i, segmento])
3     mintermino = np.arange(16) == i
4
5     if nuevo:
6         corregida[:, segmento] |= mintermino
7     else:
8         corregida[:, segmento] &= ~mintermino

```

Listing 4: Corrección de las entradas en las que falla el árbol evolucionado.

Por ello, las líneas OR M_0 y AND NOT M_1 de la consola indican, respectivamente, que se encendió d para la entrada 0 y se apagó f para la entrada 1. Las salidas finales quedaron definidas como

$$a = a_{PG} \wedge \neg M_4, \quad d = d_{PG} \vee M_0, \quad f = f_{PG} \wedge \neg M_1, \quad g = g_{PG} \wedge \neg M_7,$$

mientras que $b = b_{PG}$, $c = c_{PG}$ y $e = e_{PG}$. Finalmente, el programa volvió a evaluar las siete salidas para las 16 entradas. El mensaje 112/112 salidas correctas confirma que el circuito *evolucionado y corregido* reproduce toda la tabla de verdad; no atribuye los cuatro ajustes a la evolución por sí sola.

1.2. Ejercicio 3: detección de posibles fraudes

Se utilizó programación genética para obtener una regla que clasifica transacciones como posibles fraudes. El ejercicio comprende dos etapas: generar un conjunto de datos sintéticos y evolucionar un programa booleano a partir de esos datos. Los resultados describen el comportamiento del modelo en el conjunto generado; no constituyen una evaluación con transacciones bancarias reales.

1.2.1. Generación y preparación de los datos

Se generaron 5000 registros con cinco columnas: monto en pesos colombianos, hora del día, distancia a la residencia en kilómetros, ingreso mensual en pesos y etiqueta de fraude. Primero se asignó una clase latente con una probabilidad inicial de fraude del 9%. Después se generaron los atributos: los fraudes tuvieron mayor probabilidad de presentar montos elevados, distancias grandes y operaciones entre las 00:00 y las 05:59. Las distribuciones de ambas clases se solapan; además, se invirtió aleatoriamente el 1,2% de las etiquetas para introducir incertidumbre.

El siguiente fragmento muestra las decisiones principales de la generación. La semilla fija permite volver a producir el mismo archivo CSV:

```

1 rng = np.random.default_rng(20260929)
2 fraude = (rng.random(5000) < 0.09).astype(int)
3
4 ingreso = rng.lognormal(
5     np.log(3_400_000), 0.55, 5000
6 ).clip(950_000, 16_000_000)
7
8 hora = rng.integers(0, 24, 5000)
9 nocturno = fraude.astype(bool) & (rng.random(5000) < 0.48)
10 hora[nocturno] = rng.integers(0, 6, nocturno.sum())
11
12 monto = rng.lognormal(np.log(115_000), 1.05, 5000)

```

```

13 monto[fraude == 1] *= rng.lognormal(
14     np.log(5), 0.95, (fraude == 1).sum()
15 )
16
17 distancia = rng.exponential(12, 5000)
18 distancia[fraude == 1] = rng.exponential(
19     80, (fraude == 1).sum()
20 )
21
22 invertir = rng.random(5000) < 0.012
23 fraude[invertir] = 1 - fraude[invertir]

```

Listing 5: Generación de transacciones sintéticas.

Los valores se limitaron a rangos definidos en el programa antes de guardar el CSV. La ejecución produjo 489 registros etiquetados como fraude y 4511 como no fraude, equivalentes a una proporción final de fraude del 9,78 %. Esta proporción hace que la exactitud por sí sola sea insuficiente para evaluar el clasificador: un programa que siempre responda “no fraude” acertaría muchas filas, pero no detectaría ningún fraude.

Se construyeron cinco entradas numéricas para la PG: m , monto expresado en millones de COP; h , hora del día; d , distancia en kilómetros; s , ingreso mensual expresado en millones de COP; y $r = m/s$, relación entre monto e ingreso. El conjunto se dividió de forma estratificada en 3000 registros de entrenamiento, 1000 de validación y 1000 de prueba. La estratificación conserva aproximadamente la proporción de fraudes en las tres particiones.

```

1 X = np.column_stack((
2     monto / 1e6,
3     hora,
4     distancia,
5     ingreso / 1e6,
6     monto / ingreso,
7 ))
8
9 entrenamiento, resto = train_test_split(
10     np.arange(len(datos)),
11     test_size=0.4,
12     random_state=14,
13     stratify=y,
14 )
15
16 validacion, prueba = train_test_split(
17     resto,
18     test_size=0.5,
19     random_state=14,
20     stratify=y[resto],
21 )

```

Listing 6: Construcción de atributos y particiones estratificadas.

1.2.2. Representación y evolución del clasificador

Cada individuo de la población es un árbol que devuelve un valor booleano: 1 para posible fraude y 0 para no fraude. Sus terminales son las cinco entradas anteriores y constantes numéricas. Las funciones primitivas incluyen suma, resta, comparaciones $<$ y $>$, operadores lógicos AND, OR y NOT, y una función condicional. Se usaron árboles tipados para que las operaciones aritméticas recibieran números y las operaciones lógicas recibieran valores booleanos.

```

1 conjunto = gp.PrimitiveSetTyped(
2     "FLAG", [float] * 5, bool
3 )
4
5 conjunto.addPrimitive(
6     operator.gt, [float, float], bool, name="GT"

```

```

7 )
8 conjunto.addPrimitive(
9     operator.lt, [float, float], bool, name="LT"
10 )
11 conjunto.addPrimitive(
12     np.logical_and, [bool, bool], bool, name="AND"
13 )
14 conjunto.addPrimitive(
15     np.logical_or, [bool, bool], bool, name="OR"
16 )
17 conjunto.addPrimitive(
18     operator.add, [float, float], float, name="ADD"
19 )
20 conjunto.addPrimitive(
21     operator.sub, [float, float], float, name="SUB"
22 )

```

Listing 7: Algunas funciones primitivas del clasificador.

La aptitud combina el valor F_1 calculado en entrenamiento y una penalización pequeña por el tamaño del árbol:

$$f(P) = F_1(P) - 0,00035 |P|, \quad F_1 = \frac{2 \text{ precisión sensibilidad}}{\text{precisión} + \text{sensibilidad}}.$$

El valor F_1 considera conjuntamente las falsas alarmas y los fraudes omitidos. La penalización favorece programas más cortos cuando ofrecen resultados parecidos. En el código, un clasificador que no predice ningún fraude recibe $F_1 = 0$:

```

1 estimado = predecir(arbol, conjunto, X_entrenamiento)
2
3 aptitud = (
4     f1_score(
5         y_entrenamiento,
6         estimado,
7         zero_division=0,
8     )
9     - 0.00035 * len(arbol)
10 )

```

Listing 8: Función de aptitud utilizada durante la evolución.

Se realizaron tres corridas con semillas 121, 122 y 123. Cada una empleó 240 individuos y 90 generaciones. La selección se hizo mediante torneos de cuatro individuos, con conservación de los dos mejores, probabilidad de cruce de 0,78, probabilidad de mutación de 0,14 por descendiente y altura máxima de siete. Los árboles obtenidos se compararon mediante el F_1 de validación; las etiquetas de prueba se consultaron únicamente después de escoger el modelo.

1.2.3. Resultados e interpretación

La ejecución produjo la siguiente salida. Los valores de entrenamiento indican el ajuste a los ejemplos usados para evolucionar; los de validación permiten escoger entre las tres corridas:

```

Archivo: transacciones_sinteticas_pg.csv; filas: 5000; fraudes: 489
Particiones: entrenamiento=3000, validación=1000, prueba=1000
Semilla 121: F1 entrenamiento=0.6600; F1 validación=0.6706; nodos=55
Semilla 122: F1 entrenamiento=0.6580; F1 validación=0.6667; nodos=53
Semilla 123: F1 entrenamiento=0.6735; F1 validación=0.6946; nodos=57
F1 entrenamiento=0.6735; F1 validación=0.6946
Prueba: TN=885, FP=17, FN=32, TP=66
Precisión=0.7952; sensibilidad=0.6735; F1=0.7293; exactitud balanceada=0.8273

```

Listing 9: Resultados de las tres corridas y evaluación final.

Se escogió la corrida con semilla 123, pues alcanzó el mayor F_1 de validación: 0,6946. El programa imprimió su árbol mediante llamadas anidadas a LT, SUB y ADD. Aunque la expresión impresa ocupa una línea extensa y contiene operaciones repetidas, su simplificación algebraica produce la siguiente regla equivalente:

$$\hat{y} = \begin{cases} 1, & d + 25m - h > 52, \\ 0, & d + 25m - h \leq 52. \end{cases}$$

Se comprobó que la regla simplificada y el árbol impreso devuelven la misma clasificación para los 5000 registros generados. En esta expresión, m está en millones de COP, d en kilómetros y h es la hora numérica. La combinación es una puntuación aprendida por el programa, no una suma con significado físico. Aunque se ofrecieron el ingreso s y la relación r , el árbol seleccionado no los utiliza.

En el conjunto de prueba, TN representa transacciones legítimas clasificadas correctamente; FP, falsas alarmas; FN, fraudes omitidos; y TP, fraudes detectados. La matriz de confusión obtenida fue

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	885	17
$y = 1$	32	66

Por tanto, de los 98 fraudes presentes en prueba se detectaron 66 y se omitieron 32. La precisión fue $66/(66 + 17) = 0,7952$, mientras que la sensibilidad fue $66/(66 + 32) = 0,6735$. El valor F_1 alcanzó 0,7293 y la exactitud balanceada, 0,8273.

El experimento demuestra que la PG puede construir una regla ejecutable e interpretable a partir de los atributos proporcionados. Su alcance está limitado por el origen sintético de los datos: las relaciones que aprendió reflejan las distribuciones utilizadas para generarlos. Para evaluar un detector aplicable a transacciones reales sería necesario emplear datos reales separados en el tiempo, incorporar información contextual del cliente y considerar explícitamente el costo de las falsas alarmas y de los fraudes no detectados.